

第十二节 · 开发前的工程准备：模块、协作模式与节奏

技术文档有了， 怎么用 AI 把它做出来。

从看懂文档到动手实现——一套完整、能照着干的施工方案。

从技术文档，到代码

评审让你看懂技术方案——这节定「怎么用 AI 编程把它实现」。

01

用什么模式

02

文档怎么用

03

怎么拆任务

04

怎么和 AI 协作

05

什么节奏

接下来全是实操，不空谈。

OryxOS 已经有完整的业界调研 + 需求文档 + 技术方案。这一节，就是把这些文档变成一套能照着干的施工方案：用哪种模式、文档怎么喂、任务怎么拆、和 AI 怎么协作、按什么节奏推进。

9 个 Maven 模块速览

五大核心能力 + 支撑模块——细节在评审课，这里快速对齐。

五大核心能力

provider

对接 LLM

core

ReAct 引擎

memory

记忆

tool

工具 + 沙箱

web

对外服务

支撑模块

boot

启动

cli

命令行

storage

持久化

channel-cli

CLI 渠道

引擎是 ReAct (core)，Provider / Memory / Tool 三块能力供养它，CLI / Web 两个入口对外。

SDD 的三种落地模式

Spec-Kit / OpenSpec / 轻量手工——没有最好，只有最合适。

自动化

Spec-Kit

AI 拆任务 + 执行，你写 spec、review。适合 greenfield 中大、需求清晰、新手学方法论。

手动控制

OpenSpec

你拆任务、AI 只执行单个；delta-spec 写增量。适合 brownfield 改造、资深、系统级。

轻量手工

CLAUDE.md + Skill

不用 SDD 工具，方案在脑子里跑通，AI 一段段执行。适合熟悉的技术栈、小改动。

三种都该会，按项目类型和你的能力阶段切换用——下一页看 OryxOS 主体为什么选 Spec-Kit。

OryxOS 主体，为什么选 Spec-Kit

对照 Spec-Kit 适合的场景，OryxOS 五条全中。

- | | | |
|---|------------------------|------------------------|
| ✓ | greenfield 从零开发 | 全新项目，不是改造老代码 |
| ✓ | medium 规模、模块清晰 | 9 个 Maven 模块、五大核心能力 |
| ✓ | 需求清晰 | 已有完整需求文档 + 技术方案 |
| ✓ | AI agent 协作 | 本来就用 Claude Code 做主体开发 |
| ✓ | 方法论场景 | 强制 spec 流程，学到工程方法论 |

结论：主体开发用 Spec-Kit；增量阶段切手动提示词；OpenSpec 作对照，理解 delta 范式。

已有文档喂给 Spec-Kit, 不重写

已有投入不浪费——只是转成 AI 能直接消费的格式。

业界调研 + 需求文档

→

/specify

→

spec.md (5 个 user story)

技术方案

→

/plan

→

plan.md (9 模块+决策)

设计目标 + 关键技术决策

→

constitution

→

不可协商的项目原则

需求文档第 13 章 5 个验收 demo

→

→ 直接复用

→

5 个 US 的验收标准

关键：喂的必须是最新版文档——模块是 9 个不是 11 个，否则生成的 plan 会按错误结构走偏。

先产出三份 artifacts

一次性把地基备好，后续每个 user story 都有依据。

STEP 1

constitution

项目宪法：非协商原则，所有命令都引用

STEP 2

specify

把需求转成 5 个 user story 的 spec

STEP 3

plan

技术栈 + 9 模块职责 + 关键技术决策

三份产物放进 `.specify/`，整个主体开发期间作为依据。

plan 生成后人工 review 必做：别让 AI 把 Memory 合进 Session、把 Tool 拆成多模块、或启用 Spring AI 自动执行。

把最容易写错的，先钉死

constitution 是非协商原则——AI 每次跑都要引用它。

原则一

自实现 ReAct

ReAct 循环自己写，不用 Spring AI 的 Agent 抽象——核心完全可控。

原则二

Spring AI 只用一半

只用 Provider 抽象 + schema 生成，禁用自动 tool 执行，否则 tool 被调两次。

原则三

审计 day one 落库

tool_invocations / llm_calls 核心阶段就写库，不是只写日志。

原则四

五大能力优先

先交付运行时内核，多租户 / SSO 等治理放扩展阶段。

看到 AI 生成的代码违反这几条，立刻让它重读 constitution 改正——这是最容易踩的几个坑。

按 user story 拆，不按时间拆

5 个 US = 五大核心能力，按依赖顺序推进。



US-1 / US-2 是基础，US-3 / US-4 互不依赖可并行，US-5 收口对外暴露所有能力。

Spec-Kit 的 /tasks 本就按 user story 组织任务——OryxOS 按五大能力拆，正好顺着它的工作方式。

五个 user story 一览

每个 US：核心能力 / 涉及模块 / 对应验收 Demo。

US-1	对接 LLM	provider · core · boot	(Demo 一前置)
US-2	ReAct 循环	core · tool · channel-cli · cli	Demo 一 · 查天气穿衣
US-3	Memory 记忆	memory	Demo 二 · 跨对话记偏好
US-4	Plugin Tool	tool · core	Demo 三 · 零代码 PR digest
US-5	Web Service	web · storage · cli	Demo 四 / 五 · REST 集成

每个 user story 完成后都有一个可演示 Demo，对应需求文档第 13 章的五个验收 demo。

怎么和 AI 一起跑

Spec-Kit 跑起来后，这四条让它不漂、不偏、可回退。

防漂移

每个 US 后跑 analyze

`/speckit.analyze` 检查
constitution / spec /
plan / tasks / 代码是否
一致。

纠跑偏

违反宪法就重读

AI 生成的代码不符合
constitution，立刻让它
重读宪法改正。

防断片

上下文丢失回到 spec

跨 task 时让 AI 重读
spec.md + plan.md +
最近代码。

可回退

每个 US 一次 commit

git commit 标记每个
user story 完成，随时
回到稳定状态。

这四条是 Spec-Kit 主体开发期间的「协作纪律」——AI 负责执行，你负责守住方向。

开发节奏：4 周，每周一组能力

每周 3 小时，每周末有可演示成果——按能力依赖推进。

Week 1

对接 LLM + ReAct

搭骨架、Provider、ReAct 循环、HTTP Tool、CLI。
Demo: chat 多轮 + 调 Tool。

Week 2

Memory + Tool

MEMORY.md、内置 Tool、MCP Client、SKILL.md。
Demo: 记偏好 + 调外部 MCP。

Week 3

Web Service

Spring MVC、6 个 Controller、10 个端点。
Demo: 外部系统完整调用。

Week 4

多 Agent + 收尾

多 Agent 并存、Session 落库、12 命令、主页。
Demo: 多 Agent + 跨重启。

具体时间投入由团队定——本节给的是顺序与节奏的骨架，不是死的时间表。

施工方案有了， 可以动手了。

两阶段模式、喂文档、constitution、按 US 拆、协作纪律、4 周节奏——全部就位。

下一节

第十三节：Spec-Kit 实战——原理 + 现场拆解任务

- Spec-Kit 的原理深讲
- 现场装好、把 OryxOS 文档喂进去
- 跑 specify → plan → tasks，把需求拆成任务